

Introduction to Seaborn for Visualization



seaborn

✓ Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ Import Data

The `seaborn` library has a set of sample datasets available for producing examples and testing in its [online repository](#).

We will be using `seaborn`'s `.load_dataset` API to fetch the data we want. We will need access to the internet to use this API.

To see a list of the available dataset, we use `sns.get_dataset_names()`.

```
sns.get_dataset_names()
```

```
['anagrams',
 'anscombe',
 'attention',
 'brain_networks',
 'car_crashes',
 'diamonds',
 'dots',
 'dowjones',
 'exercise',
 'flights',
 'fmri',
 'geyser',
 'glue',
 'healthexp',
 'iris',
 'mpg',
 'penguins',
 'planets',
 'seaice',
 'taxis',
 'tips',
 'titanic']
```

Datasets used:

1. `car_crashes`
2. `tips`
3. `flights`
4. `iris`

We will be using the `car_crashes` dataset which contains information about car crashes in the USA. Each row represents information about car crashes in each state of the USA.

We will also use the `tips` dataset which represents some tipping data where one waiter recorded information about each tip he received over a period of a few months working in one restaurant. In all the waiter recorded 244 tips. The waiter collected several variables:

- the tip in dollars
- the bill in dollars
- the sex of the bill payer
- whether there were smokers in the party
- the day of the week
- the time of day
- the size of the party

The `flights` dataset from the same library is based off of years and months of how many people flew an airline.

The `iris` dataset consists of 50 samples from each of three species of Iris (Iris setosa, Iris virginica and Iris versicolor). The Iris dataset was used in R.A. Fisher's classic 1936 paper, The Use of Multiple Measurements in Taxonomic Problems, and can also be found on the UCI Machine Learning Repository. Features include:

- Id
- SepalLengthCm
- SepalWidthCm
- PetalLengthCm
- PetalWidthCm
- Species

Load First Data

```
crash_df = sns.load_dataset('car_crashes')
crash_df.head()
```

	total	speeding	alcohol	not_distracted	no_previous	ins_premium	ins_losses	abbrev
0	18.8	7.332	5.640	18.048	15.040	784.55	145.08	AL
1	18.1	7.421	4.525	16.290	17.014	1053.48	133.93	AK
2	18.6	6.510	5.208	15.624	17.856	899.47	110.35	AZ
3	22.4	4.032	5.824	21.056	21.280	827.34	142.39	AR
4	12.0	4.200	3.360	10.920	10.680	878.41	165.63	CA

```
crash_df.shape
```

(51, 8)

```
crash_df.describe()
```

	total	speeding	alcohol	not_distracted	no_previous	ins_premium	ins_losses
count	51.000000	51.000000	51.000000	51.000000	51.000000	51.000000	51.000000
mean	15.790196	4.998196	4.886784	13.573176	14.004882	886.957647	134.493137
std	4.122002	2.017747	1.729133	4.508977	3.764672	178.296285	24.835922
min	5.900000	1.792000	1.593000	1.760000	5.900000	641.960000	82.750000
25%	12.750000	3.766500	3.894000	10.478000	11.348000	768.430000	114.645000
50%	15.600000	4.608000	4.554000	13.857000	13.775000	858.970000	136.050000
75%	18.500000	6.439000	5.604000	16.140000	16.755000	1007.945000	151.870000
max	23.900000	9.450000	10.038000	23.661000	21.280000	1301.520000	194.780000

Second dataset

```
tips = sns.load_dataset('tips')
tips.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
tips.shape
```

```
(244, 7)
```

```
tips.describe()
```

	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

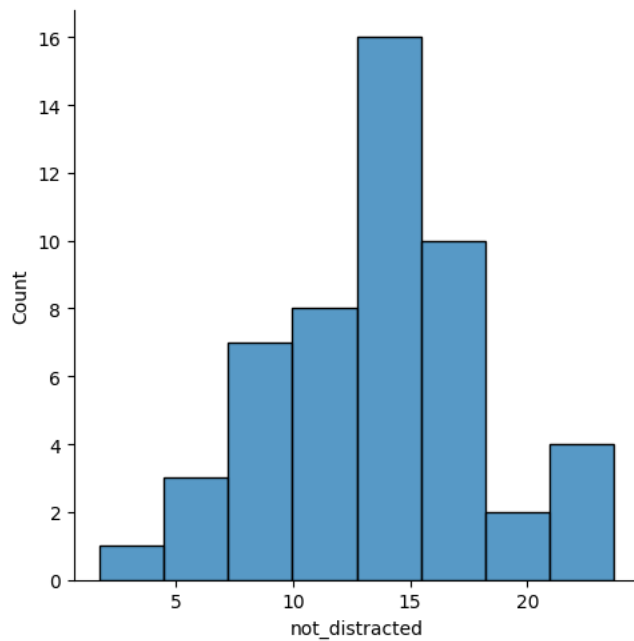
✧ Distribution Plots for Continous Variables

Note!!!

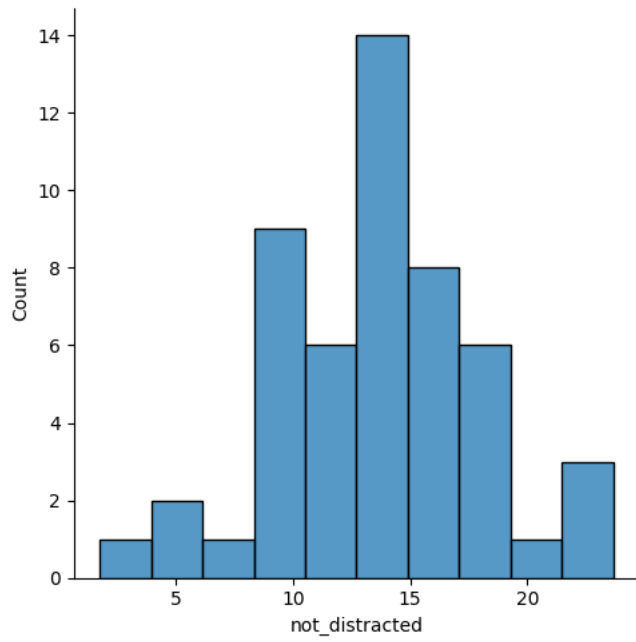
The `sns.distplot` is a deprecated function and will be removed in seaborn v0.14.0.

✧ Distribution Plot

```
sns.displot(crash_df['not_distracted']);
```

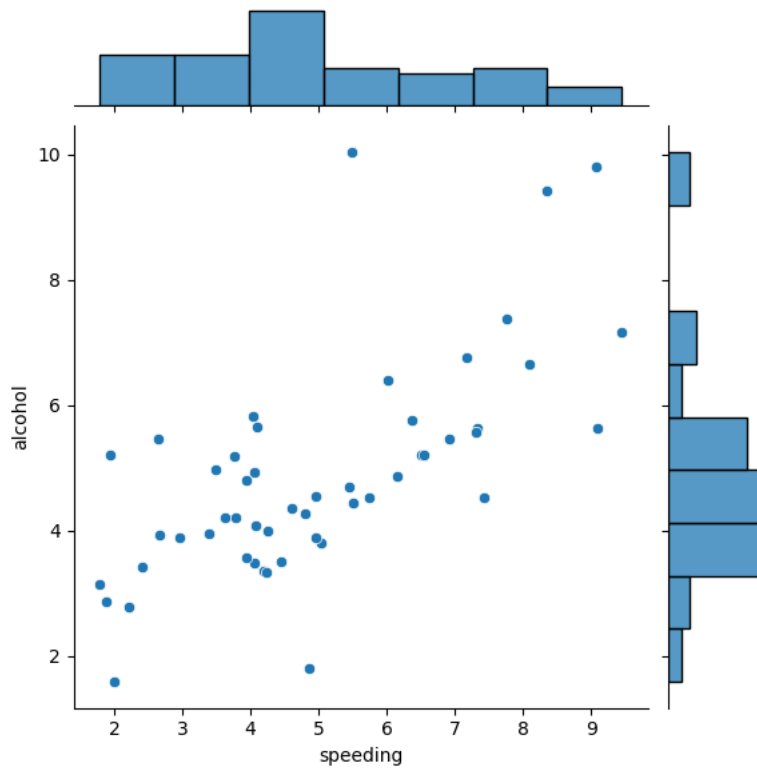


```
sns.displot(crash_df['not_distracted'], bins=10);
```

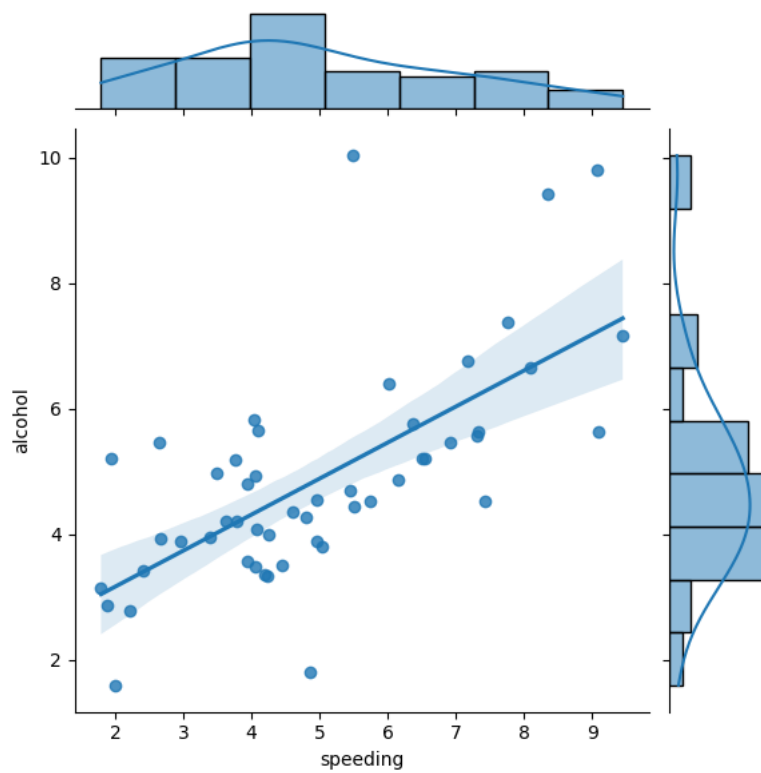


- Joint Plot

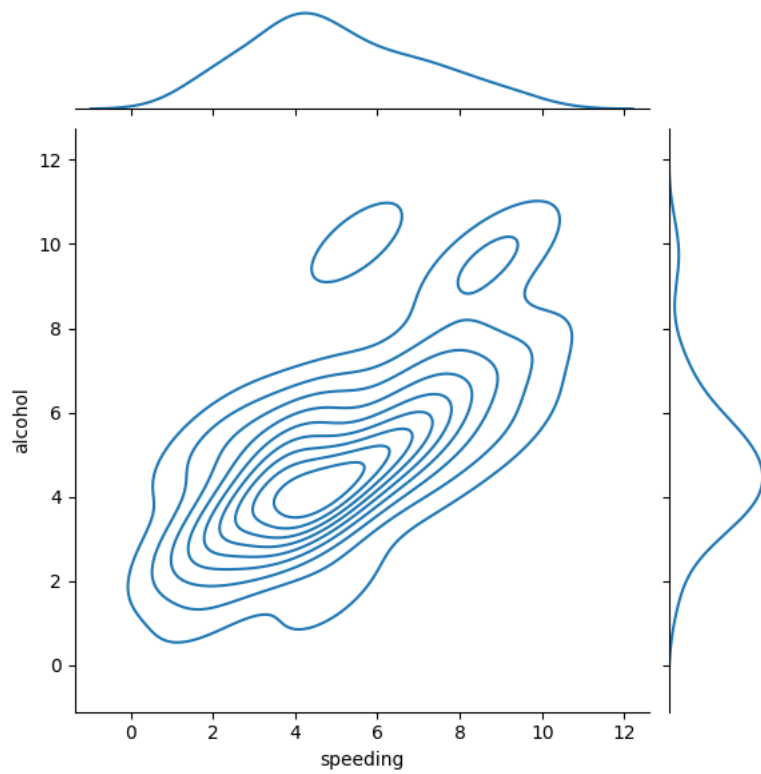
```
sns.jointplot(x='speeding',
              y='alcohol',
              data=crash_df);
```



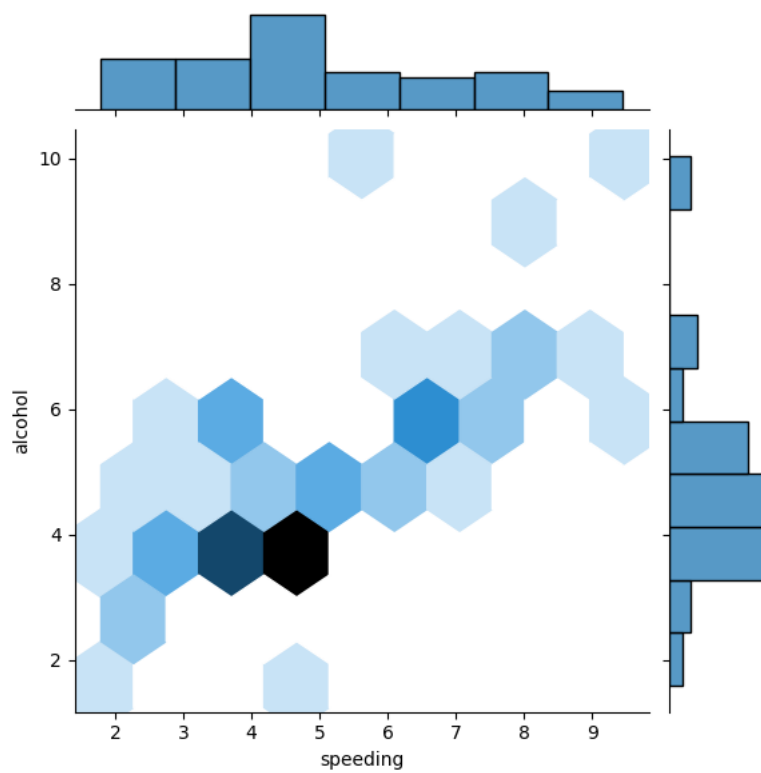
```
sns.jointplot(x='speeding',
              y='alcohol',
              data=crash_df,
              kind='reg');
```



```
sns.jointplot(x='speeding',
              y='alcohol',
              data=crash_df,
              kind='kde');
```

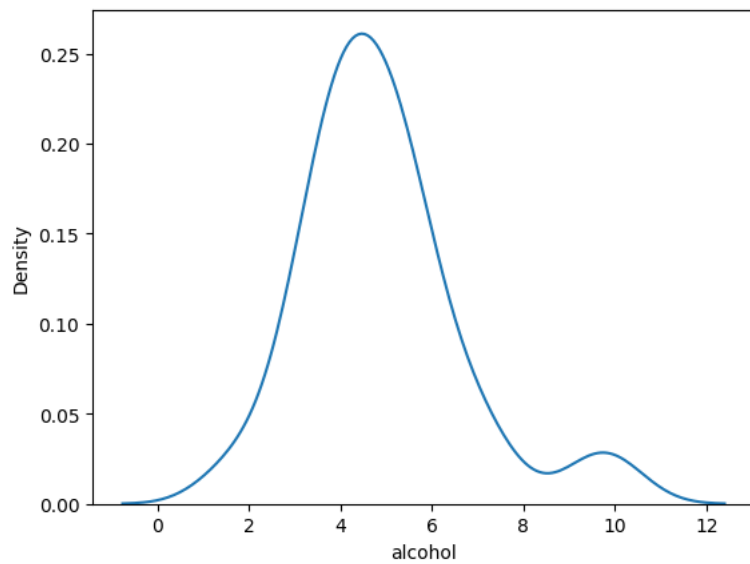


```
sns.jointplot(x='speeding',
              y='alcohol',
              data=crash_df,
              kind='hex');
```

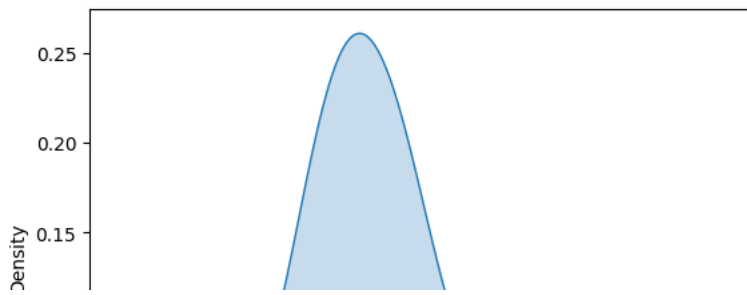


✓ KDE Plot

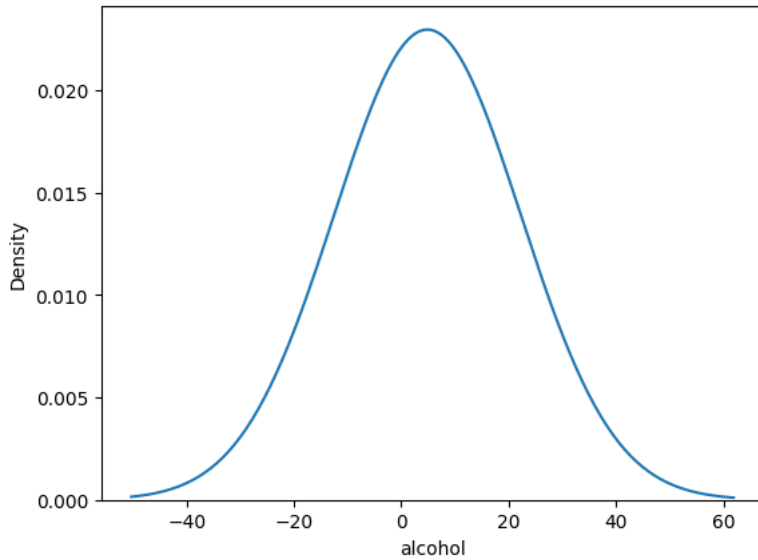
```
sns.kdeplot(crash_df['alcohol']);
```



```
sns.kdeplot(crash_df['alcohol'], fill=True);
```

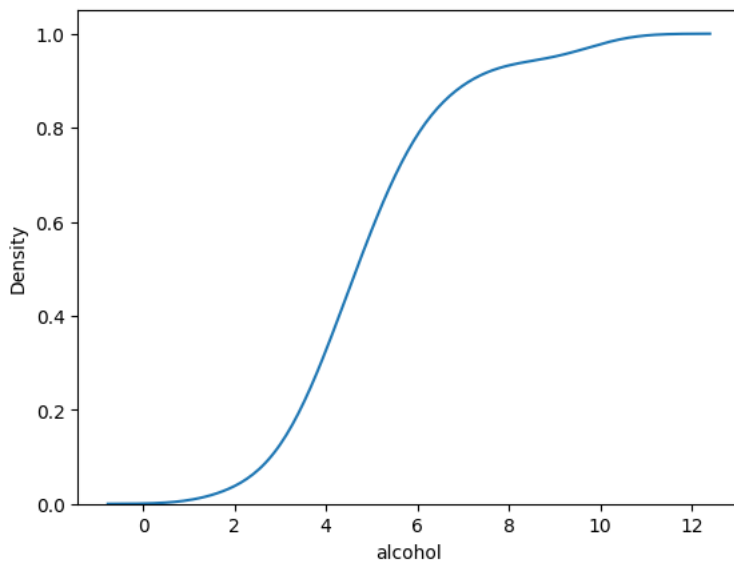


```
sns.kdeplot(crash_df['alcohol'], bw_method=10);
```



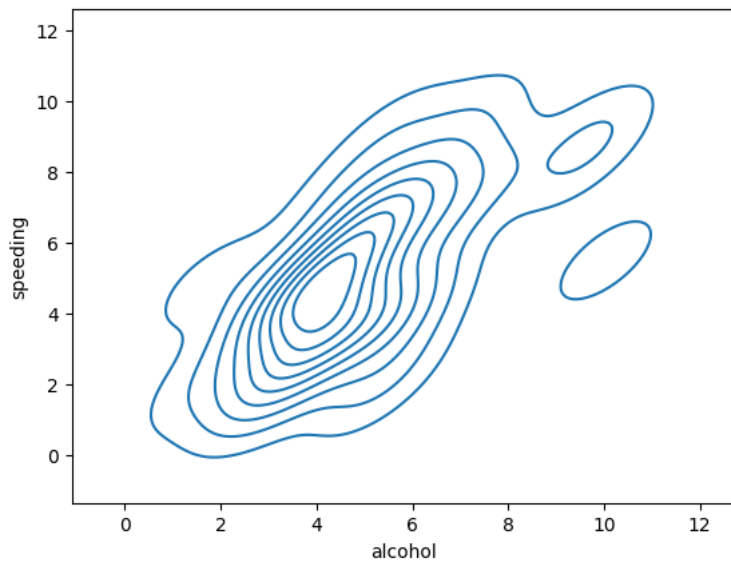
```
# Switch from Probability Density Function (PDF)
# to Cumulative Frequency Distribution (CDF)
```

```
sns.kdeplot(crash_df['alcohol'], cumulative=True);
```

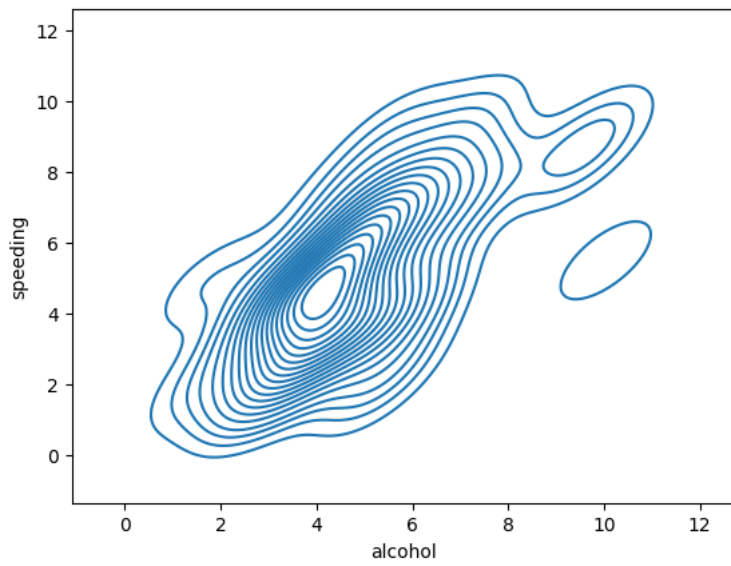


▼ Bivariate Analysis with KDE Plot

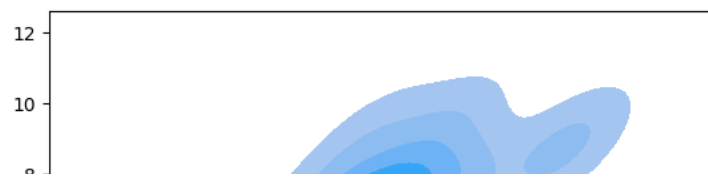
```
sns.kdeplot(x='alcohol',
            y='speeding',
            data=crash_df);
```



```
sns.kdeplot(x='alcohol',  
            y='speeding',  
            data=crash_df,  
            n_levels=20);
```



```
sns.kdeplot(x='alcohol',  
            y='speeding',  
            data=crash_df,  
            fill=True);
```

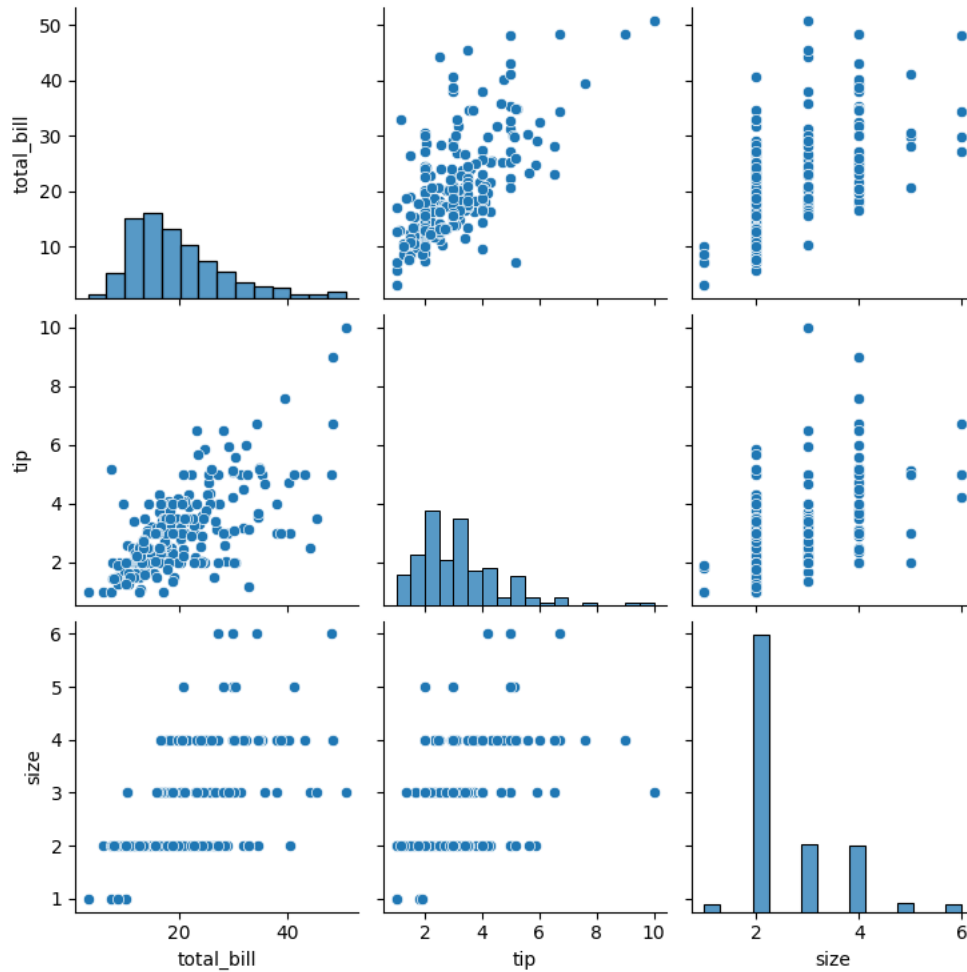



✓ Pair Plot



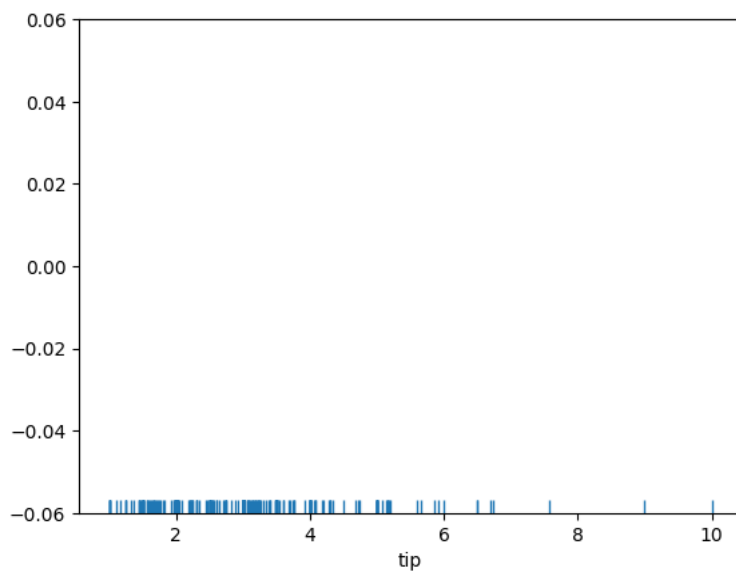
```
sns.pairplot(crash_df);
```

```
sns.pairplot(tips);
```



✓ Rug Plot

```
sns.rugplot(tips['tip']);
```

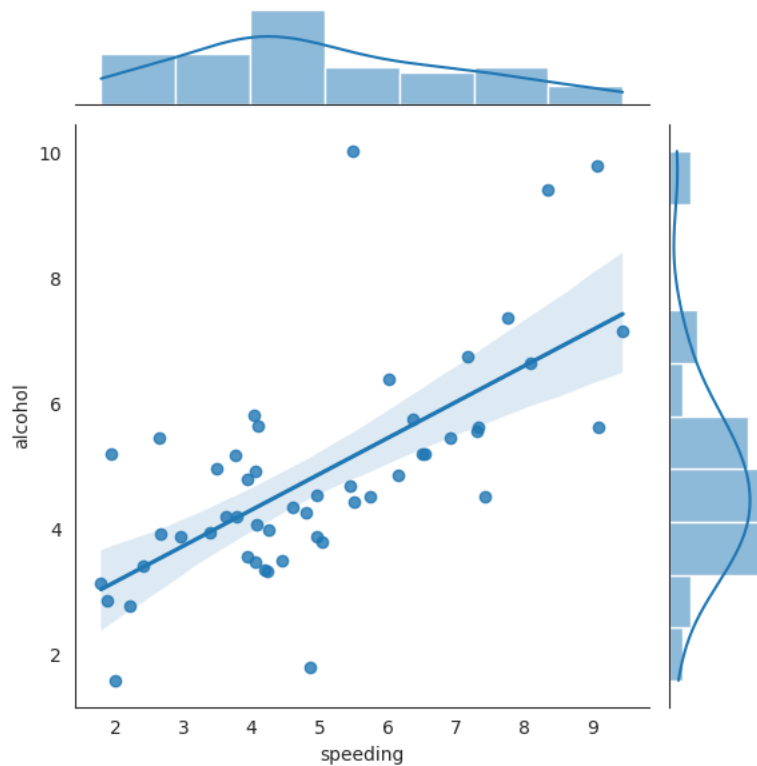


✓ Styling Plots in Seaborn

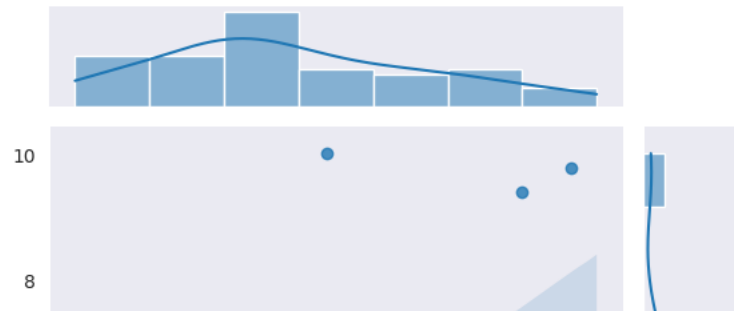
Available seaborn styles are: darkgrid, whitegrid, dark, white and ticks.

Using the `sns.axes_style` function as a context manager, we can temporarily change the style of your plots:

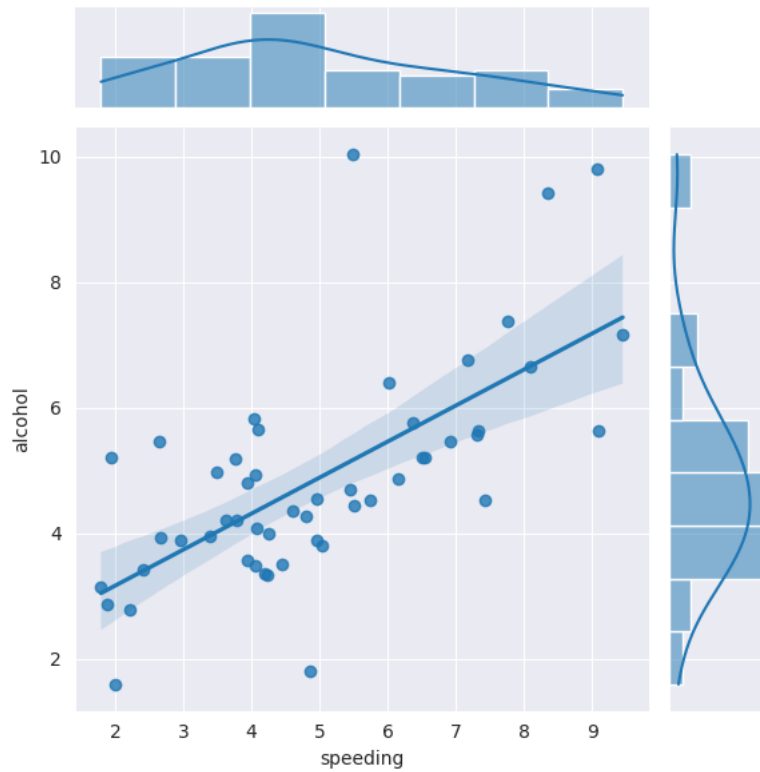
```
with sns.axes_style('white'):  
    sns.jointplot(x='speeding',  
                  y='alcohol',  
                  data=crash_df,  
                  kind='reg'  
                  );
```



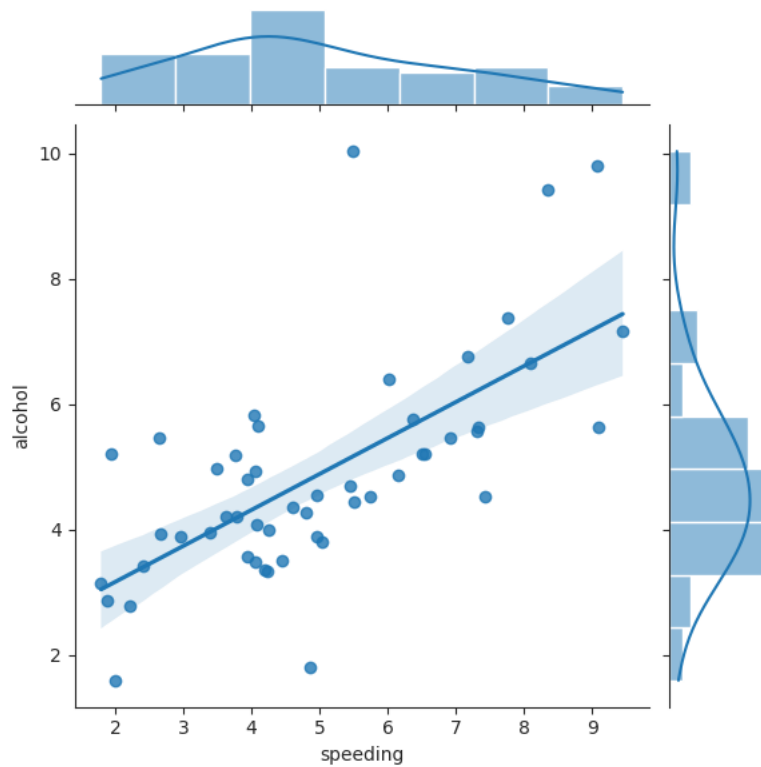
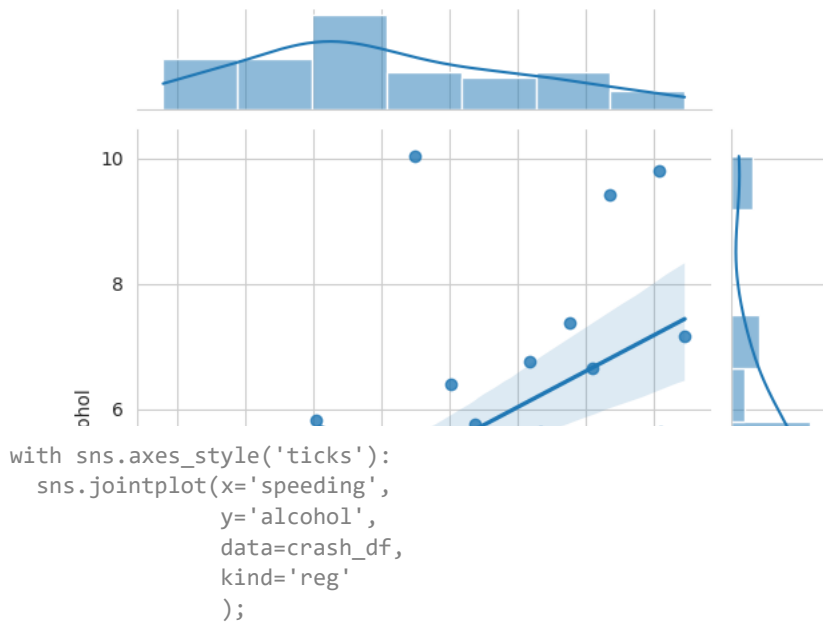
```
with sns.axes_style('dark'):  
    sns.jointplot(x='speeding',  
                  y='alcohol',  
                  data=crash_df,  
                  kind='reg'  
                  );
```



```
with sns.axes_style('darkgrid'):
    sns.jointplot(x='speeding',
                  y='alcohol',
                  data=crash_df,
                  kind='reg'
                  );
```



```
with sns.axes_style('whitegrid'):
    sns.jointplot(x='speeding',
                  y='alcohol',
                  data=crash_df,
                  kind='reg'
                  );
```



The `set_context` function allows us to control the scale of plot elements.

The four preset contexts, in order of relative size, are `paper`, `notebook`, `talk`, and `poster`.

The `notebook` style is the default, and was used in the plots above.

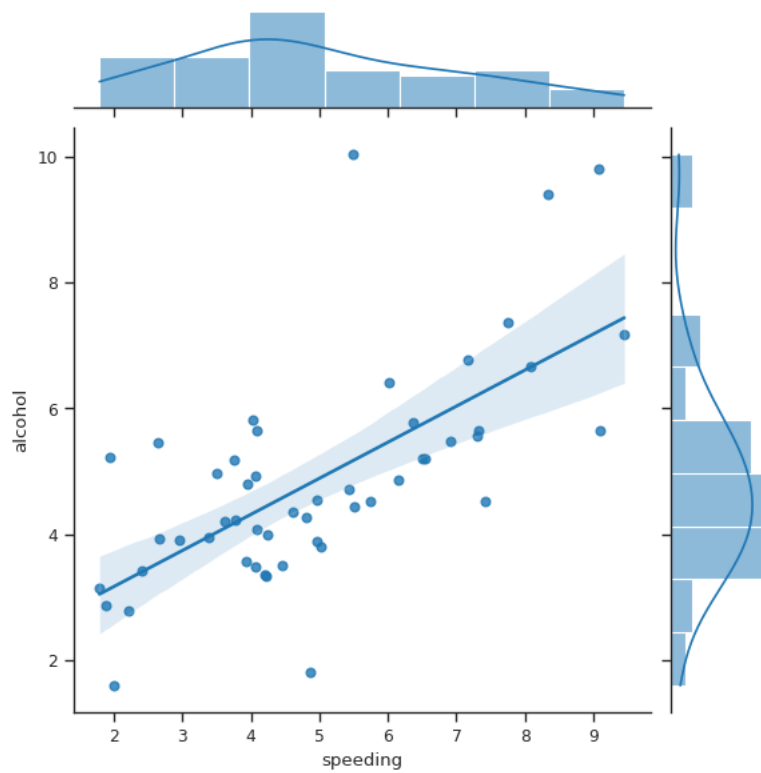
```
sns.set_style('ticks')
```

```
sns.set_context("paper")
```

```

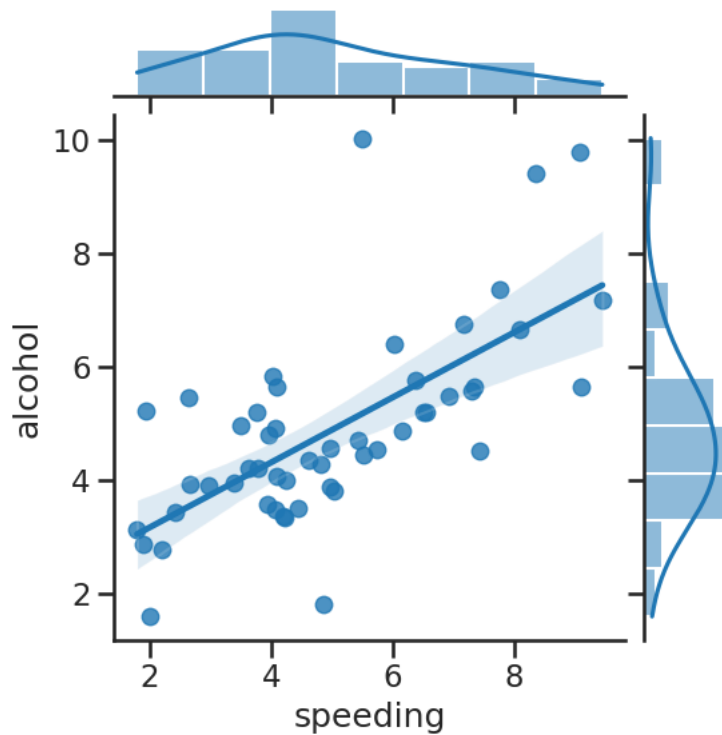
sns.jointplot(x='speeding',
              y='alcohol',
              data=crash_df,
              kind='reg'
              );

```



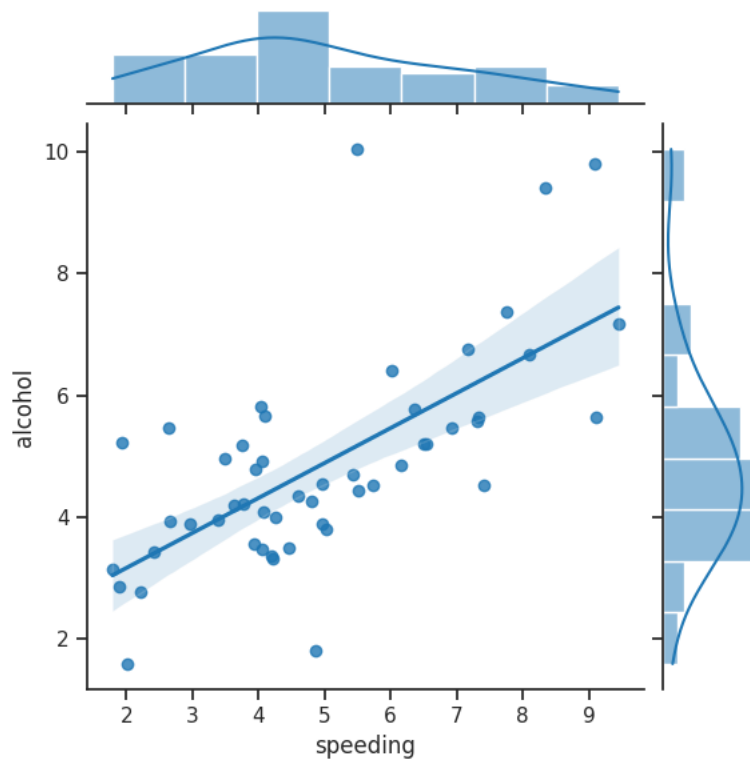
```
sns.set_context("talk")
```

```
sns.jointplot(x='speeding',
              y='alcohol',
              data=crash_df,
              kind='reg'
              );
```



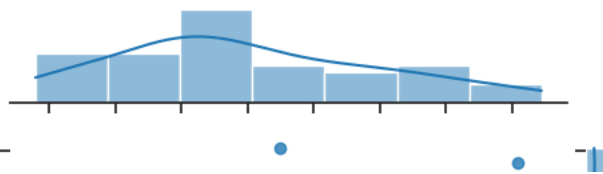
```
sns.set_context("notebook")
```

```
sns.jointplot(x='speeding',  
             y='alcohol',  
             data=crash_df,  
             kind='reg'  
             );
```

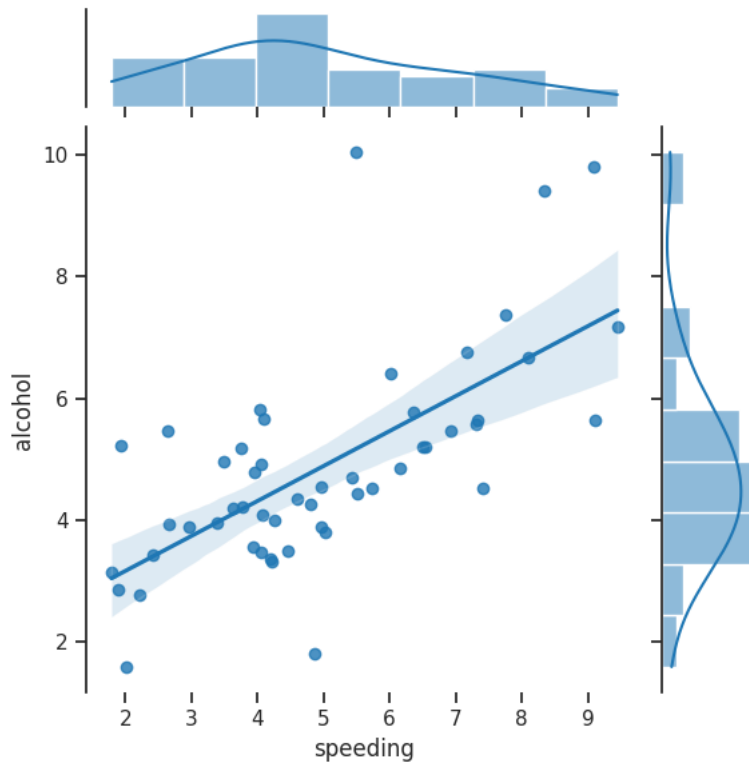


Remove spines from plot

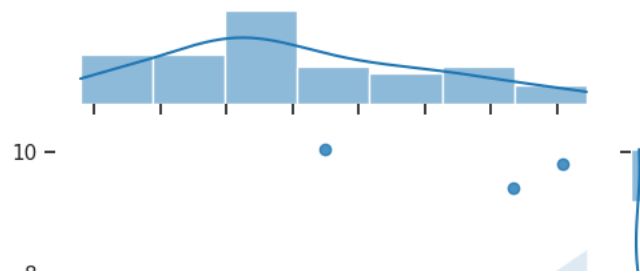
```
sns.jointplot(x='speeding',  
             y='alcohol',  
             data=crash_df,  
             kind='reg'  
             );  
sns.despine(left=True)
```



```
sns.jointplot(x='speeding',
              y='alcohol',
              data=crash_df,
              kind='reg'
            );
sns.despine(bottom=True)
```



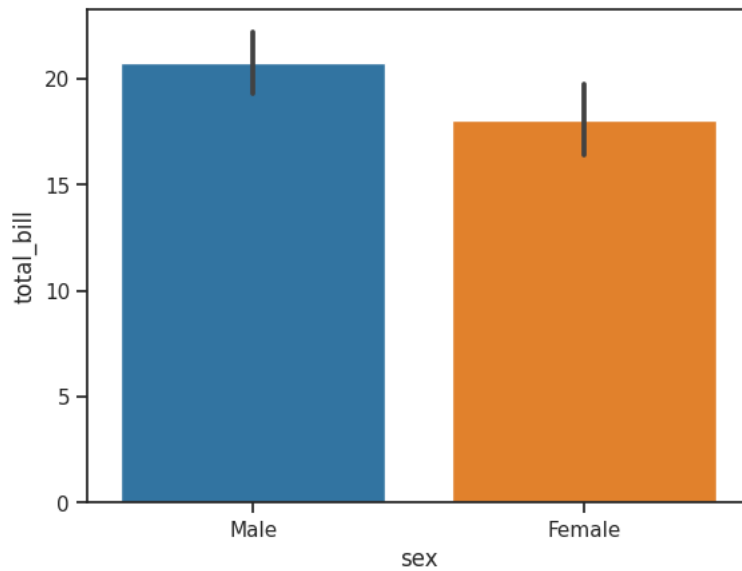
```
sns.jointplot(x='speeding',
              y='alcohol',
              data=crash_df,
              kind='reg'
            );
sns.despine(left=True, bottom=True)
```

✓ Categorical Plots for categorical data

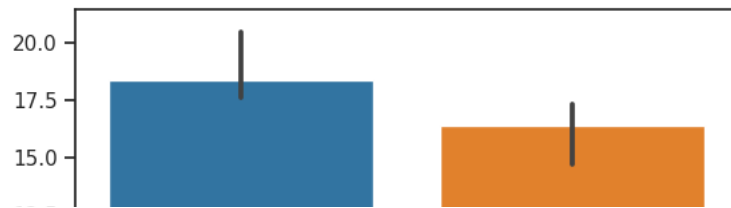
✓ Bar plot

```
sns.barplot(x='sex',
            y='total_bill',
            data=tips
            );
```

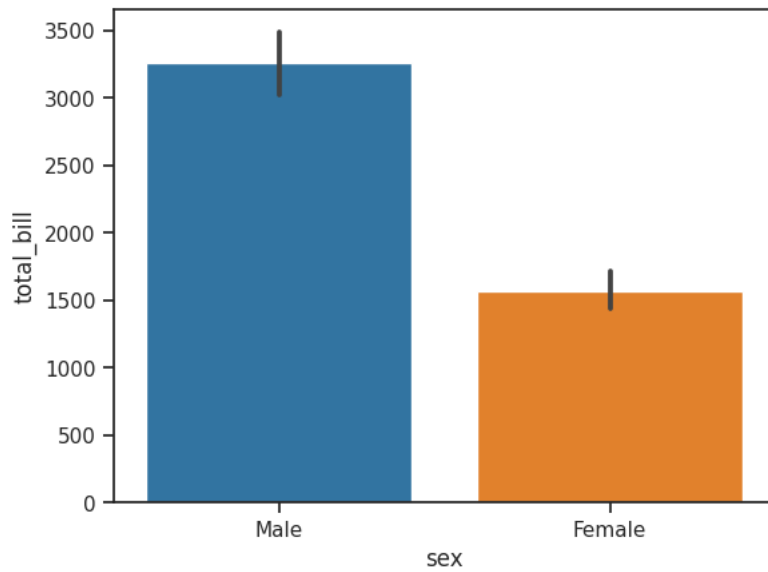


The default estimator for the `sns.barplot` function is the **mean**. We can change this by using the `estimator` parameter. We pass an **aggregation** function as a string to the parameter.

```
sns.barplot(x='sex',
            y='total_bill',
            data=tips,
            estimator='median'
            );
```

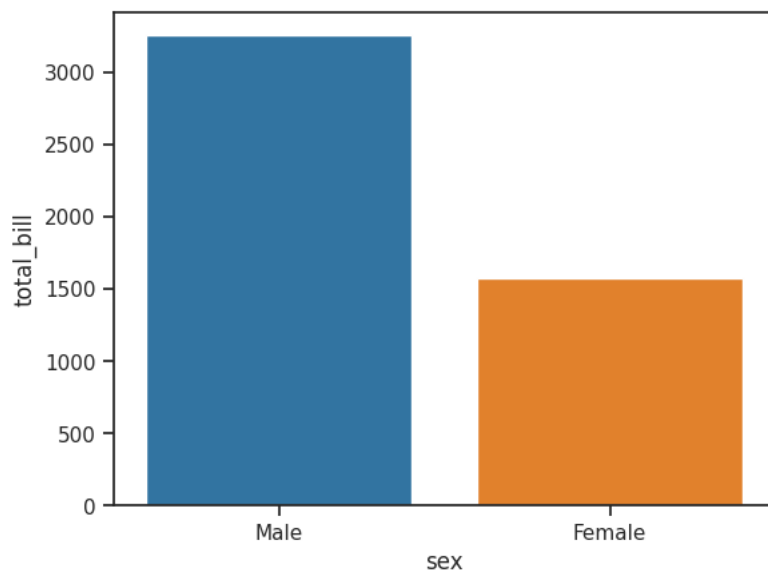


```
sns.barplot(x='sex',
            y='total_bill',
            data=tips,
            estimator='sum'
            );
```



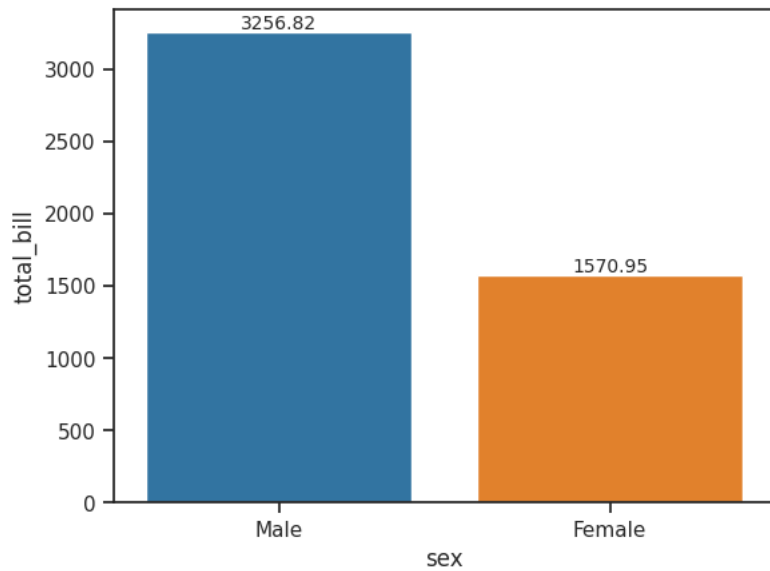
Remove error bars

```
sns.barplot(x='sex',
            y='total_bill',
            data=tips,
            estimator='sum',
            errorbar=None
            );
```



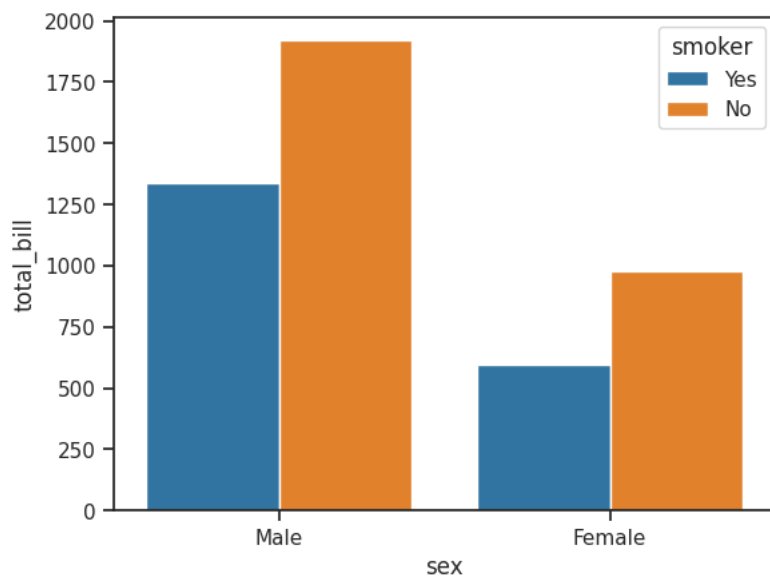
Add Data Labels

```
ax = sns.barplot(x='sex',  
                 y='total_bill',  
                 data=tips,  
                 estimator='sum',  
                 errorbar=None,  
                 );  
  
ax.bar_label(ax.containers[0], fontsize=10);
```



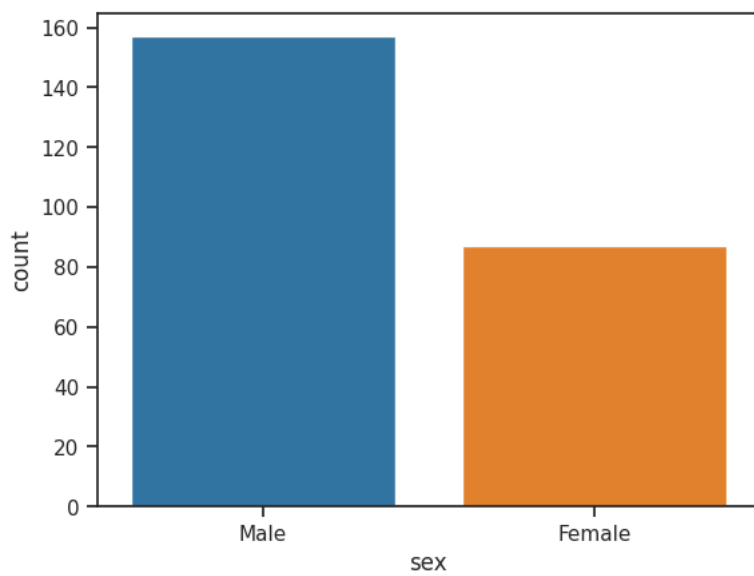
Clustered bar chart

```
sns.barplot(x='sex',  
            y='total_bill',  
            data=tips,  
            estimator='sum',  
            errorbar=None,  
            hue='smoker',  
            );
```



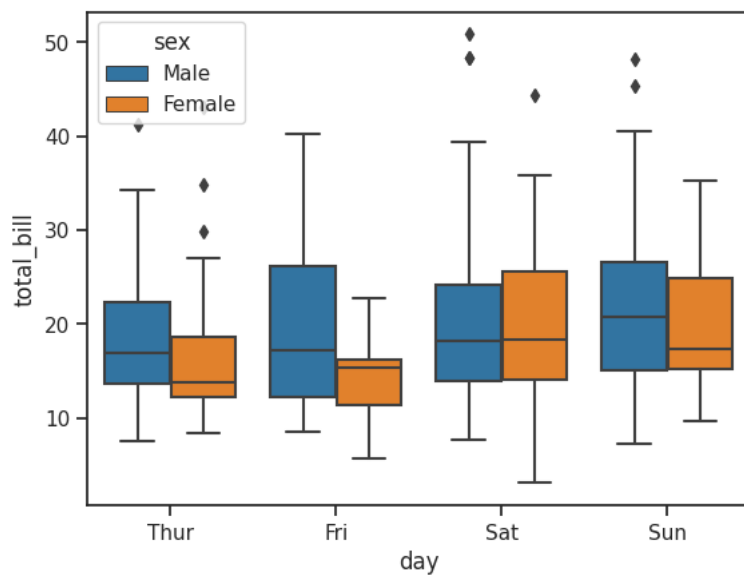
✓ Count plot

```
sns.countplot(x='sex', data=tips);
```



✓ Box plot

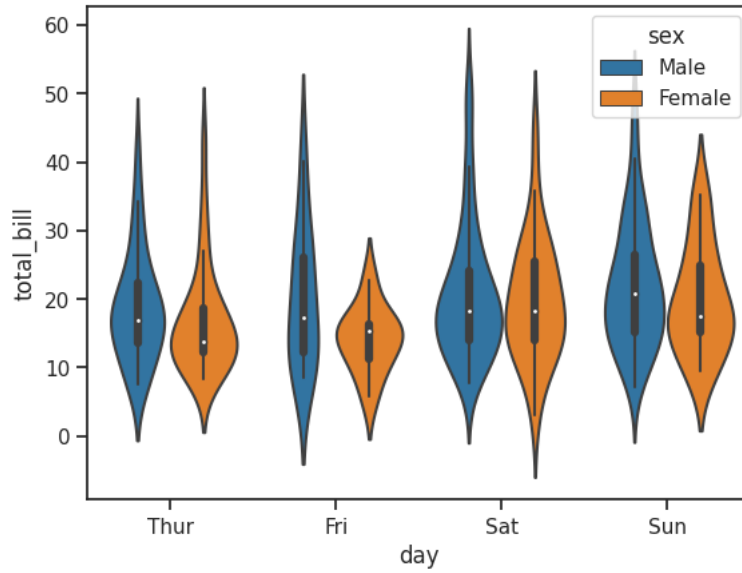
```
sns.boxplot(x='day',  
            y='total_bill',  
            hue='sex',  
            data=tips  
            );
```



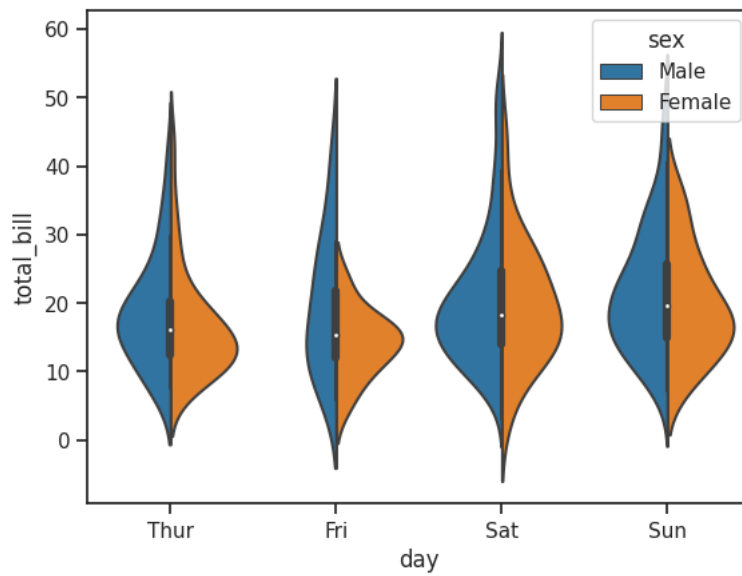
✓ Violin plot

The violin plot is a combination of a box plot and a KDE plot.

```
sns.violinplot(x='day',
               y='total_bill',
               hue='sex',
               data=tips
               );
```



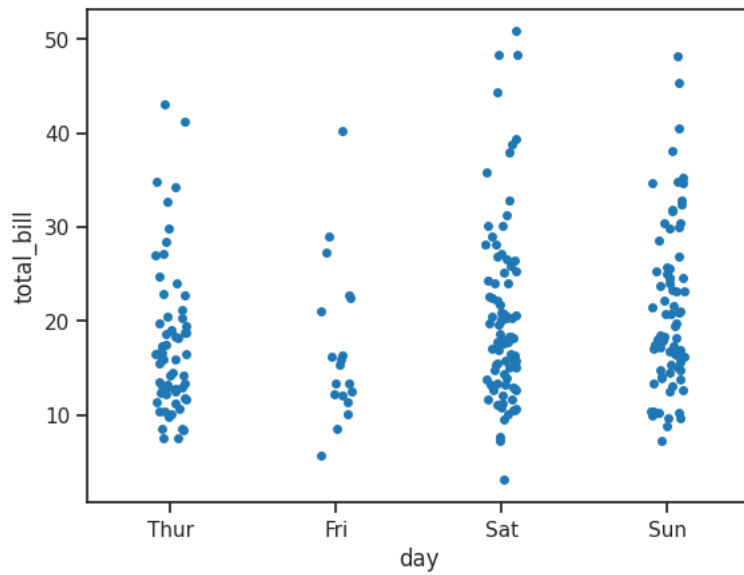
```
sns.violinplot(x='day',
               y='total_bill',
               hue='sex',
               data=tips,
               split=True
               );
```



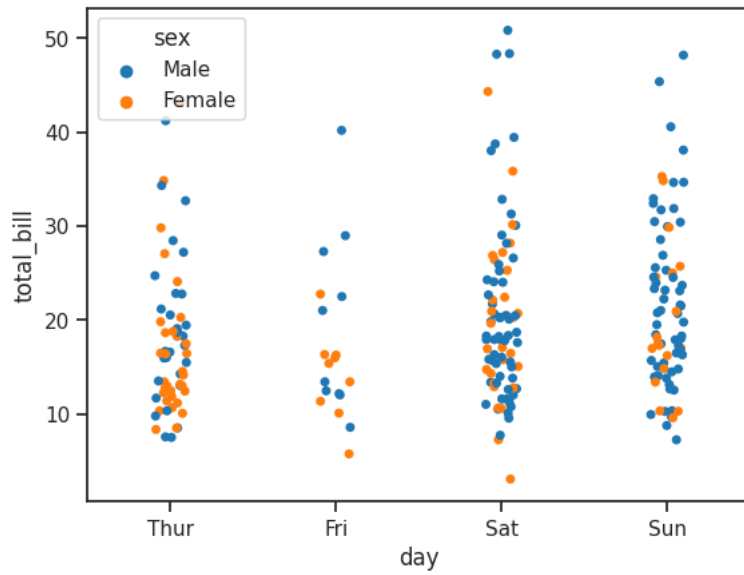
▾ Strip Plot

A strip plot plots scatter plots representing all data points for a continuous feature across categories of a categorical feature.

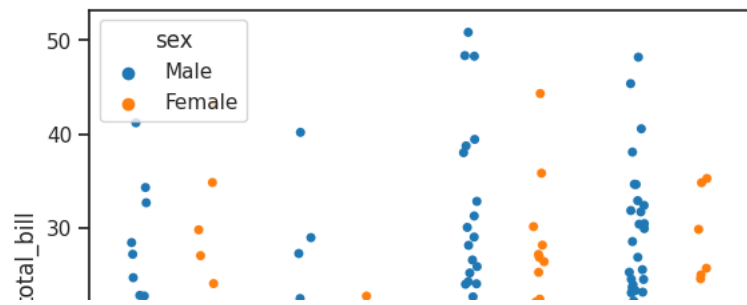
```
sns.stripplot(x='day',
              y='total_bill',
              data=tips
              );
```



```
sns.stripplot(x='day',
              y='total_bill',
              hue='sex',
              data=tips
              );
```



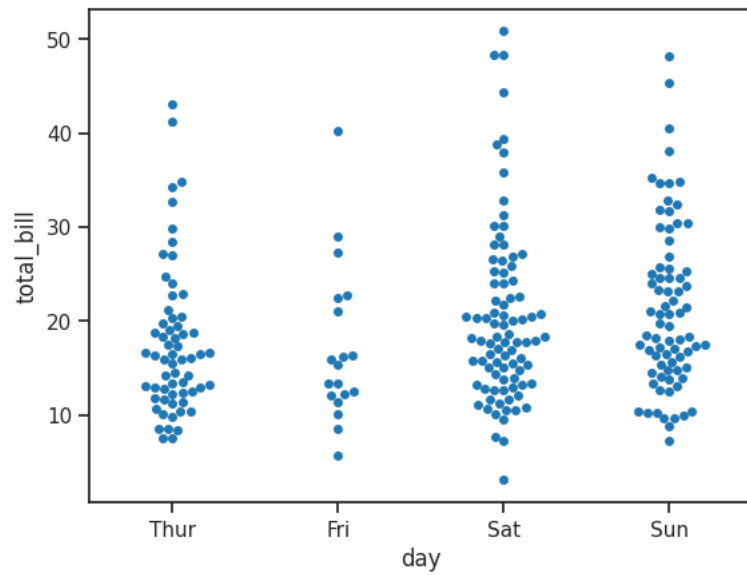
```
sns.stripplot(x='day',
              y='total_bill',
              hue='sex',
              data=tips,
              dodge=True
              );
```



✓ Swarm Plot



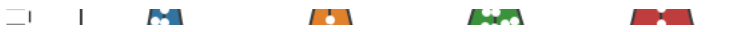
```
sns.swarmplot(x='day',
              y='total_bill',
              data=tips,
              );
```



```
sns.violinplot(x='day',
               y='total_bill',
               data=tips,
               )
```

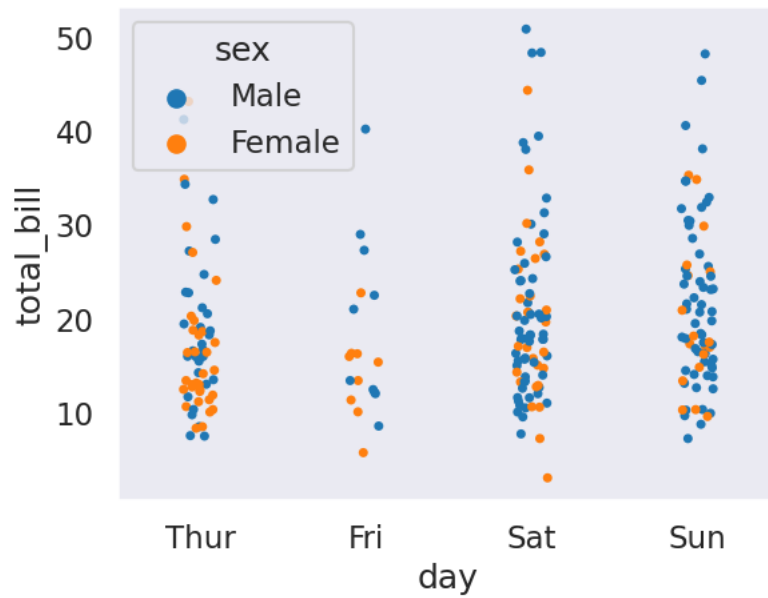
```
sns.swarmplot(x='day',
              y='total_bill',
              data=tips,
              color='white'
              );
```

Palettes



```
sns.set_context('talk')
```

```
with sns.axes_style('dark'):
    sns.stripplot(x='day',
                  y='total_bill',
                  hue='sex',
                  data=tips
                  );
```

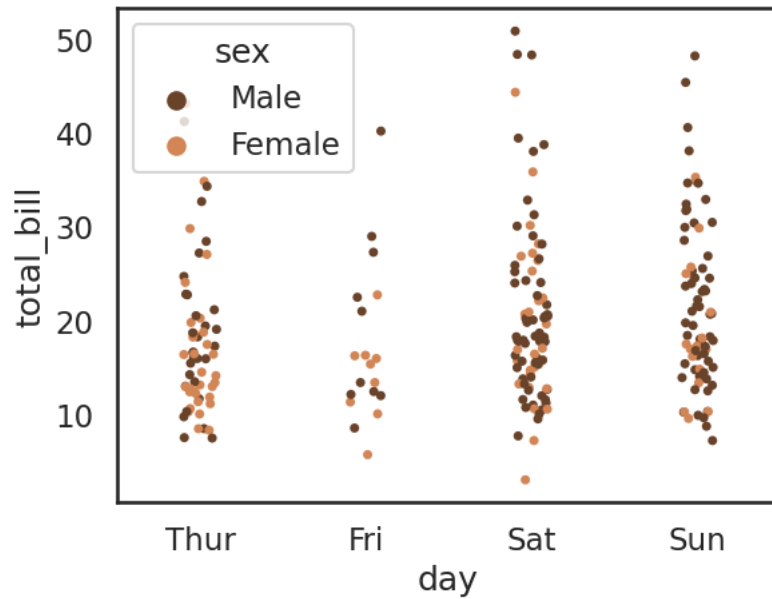


```
with sns.axes_style('dark'):
    sns.stripplot(x='day',
                  y='total_bill',
                  hue='sex',
                  data=tips,
                  palette='magma'
                  );
```


50



```
with sns.axes_style('white'):\nsns.stripplot(x='day',\n               y='total_bill',\n               hue='sex',\n               data=tips,\n               palette='copper'\n               );
```



Tweaking the position of the legend

```
with sns.axes_style('white'):\nsns.stripplot(x='day',\n               y='total_bill',\n               hue='sex',\n               data=tips,\n               palette='copper'\n               );\n\nplt.legend(loc='best', bbox_to_anchor=(1, 1));
```

50

Male

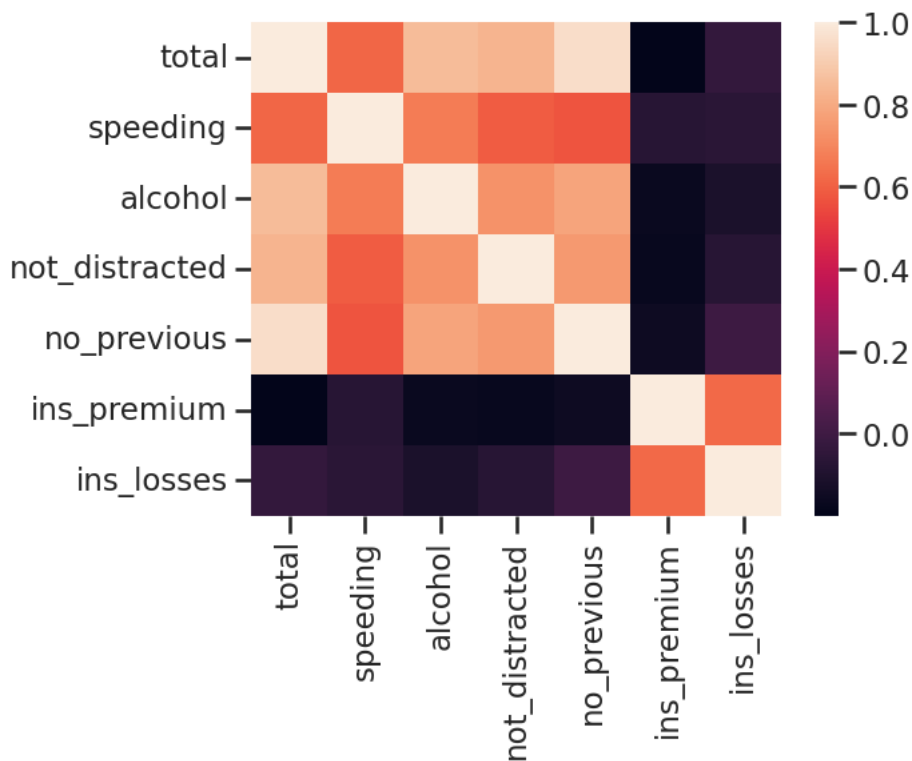
Matrix Plots

Heatmap

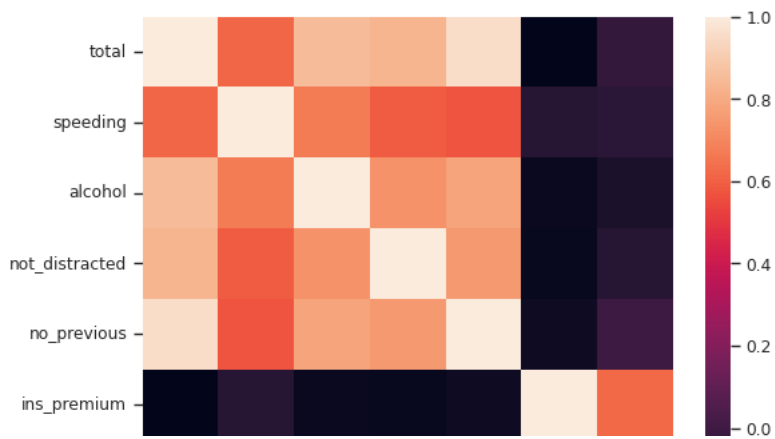
```
crash_corr = crash_df.corr(numeric_only=True)
crash_corr
```

	total	speeding	alcohol	not_distracted	no_previous	ins_premium	ins_losses
total	1.000000	0.611548	0.852613	0.827560	0.956179	-0.199702	-0.036011
speeding	0.611548	1.000000	0.669719	0.588010	0.571976	-0.077675	-0.065928
alcohol	0.852613	0.669719	1.000000	0.732816	0.783520	-0.170612	-0.112547
not_distracted	0.827560	0.588010	0.732816	1.000000	0.747307	-0.174856	-0.075970
no_previous	0.956179	0.571976	0.783520	0.747307	1.000000	-0.156895	-0.006359
ins_premium	-0.199702	-0.077675	-0.170612	-0.174856	-0.156895	1.000000	0.623116
ins_losses	-0.036011	-0.065928	-0.112547	-0.075970	-0.006359	0.623116	1.000000

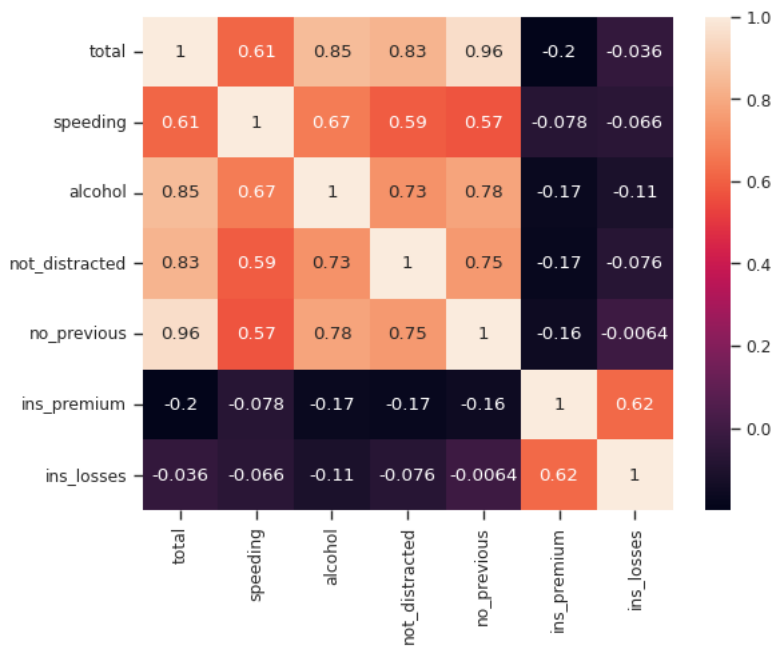
```
sns.heatmap(crash_corr);
```



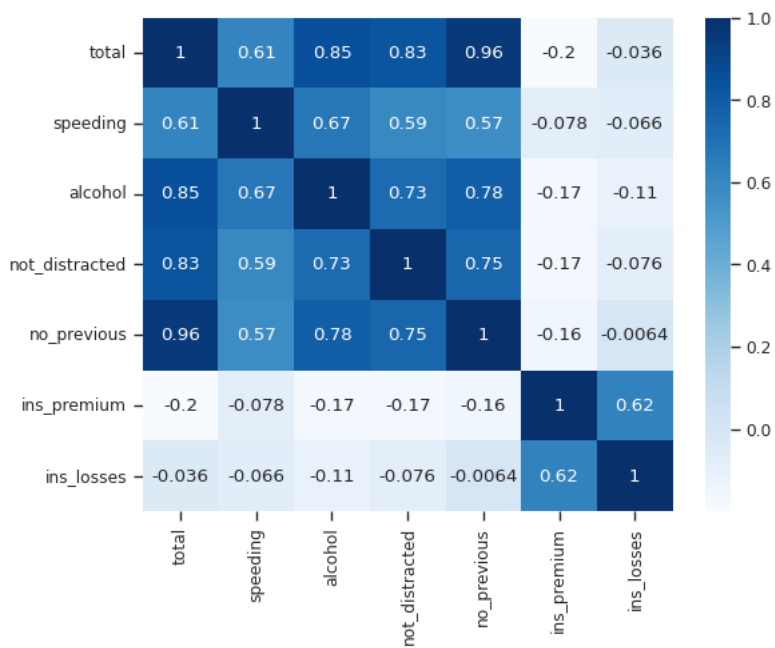
```
sns.set_context('paper')
sns.heatmap(crash_corr);
```



```
sns.heatmap(crash_corr, annot=True);
```



```
sns.heatmap(crash_corr, annot=True, cmap='Blues');
```



```
flights = sns.load_dataset('flights')
flights.head()
```

	year	month	passengers
0	1949	Jan	112
1	1949	Feb	118
2	1949	Mar	132
3	1949	Apr	129
4	1949	May	121

```
flights.shape

(144, 3)
```

```
flights_ct = flights.pivot_table(
    index='month',
    columns='year',
    values='passengers'
)
```

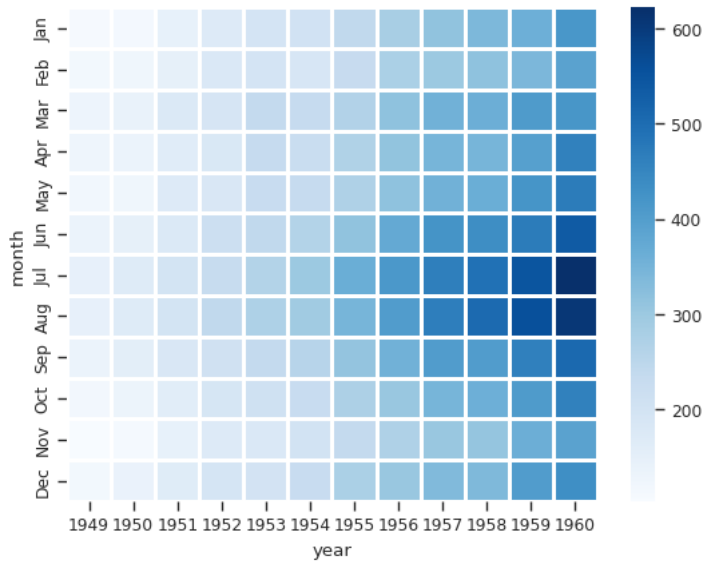
flights_ct

	year	1949	1950	1951	1952	1953	1954	1955	1956	1957	1958	1959	1960
month													
Jan	112	115	145	171	196	204	242	284	315	340	360	417	
Feb	118	126	150	180	196	188	233	277	301	318	342	391	
Mar	132	141	178	193	236	235	267	317	356	362	406	419	
Apr	129	135	163	181	235	227	269	313	348	348	396	461	
May	121	125	172	183	229	234	270	318	355	363	420	472	
Jun	135	149	178	218	243	264	315	374	422	435	472	535	
Jul	148	170	199	230	264	302	364	413	465	491	548	622	
Aug	148	170	199	242	272	293	347	405	467	505	559	606	
Sep	136	158	184	209	237	259	312	355	404	404	463	508	
Oct	119	133	162	191	211	229	274	306	347	359	407	461	
Nov	104	114	146	172	180	203	237	271	305	310	362	390	
Dec	118	140	166	194	201	229	278	306	336	337	405	432	

```
sns.heatmap(flights_ct, cmap='Blues');
```



```
sns.heatmap(
    flights_ct,
    cmap='Blues',
    linecolor='white',
    linewidth=1
);
```

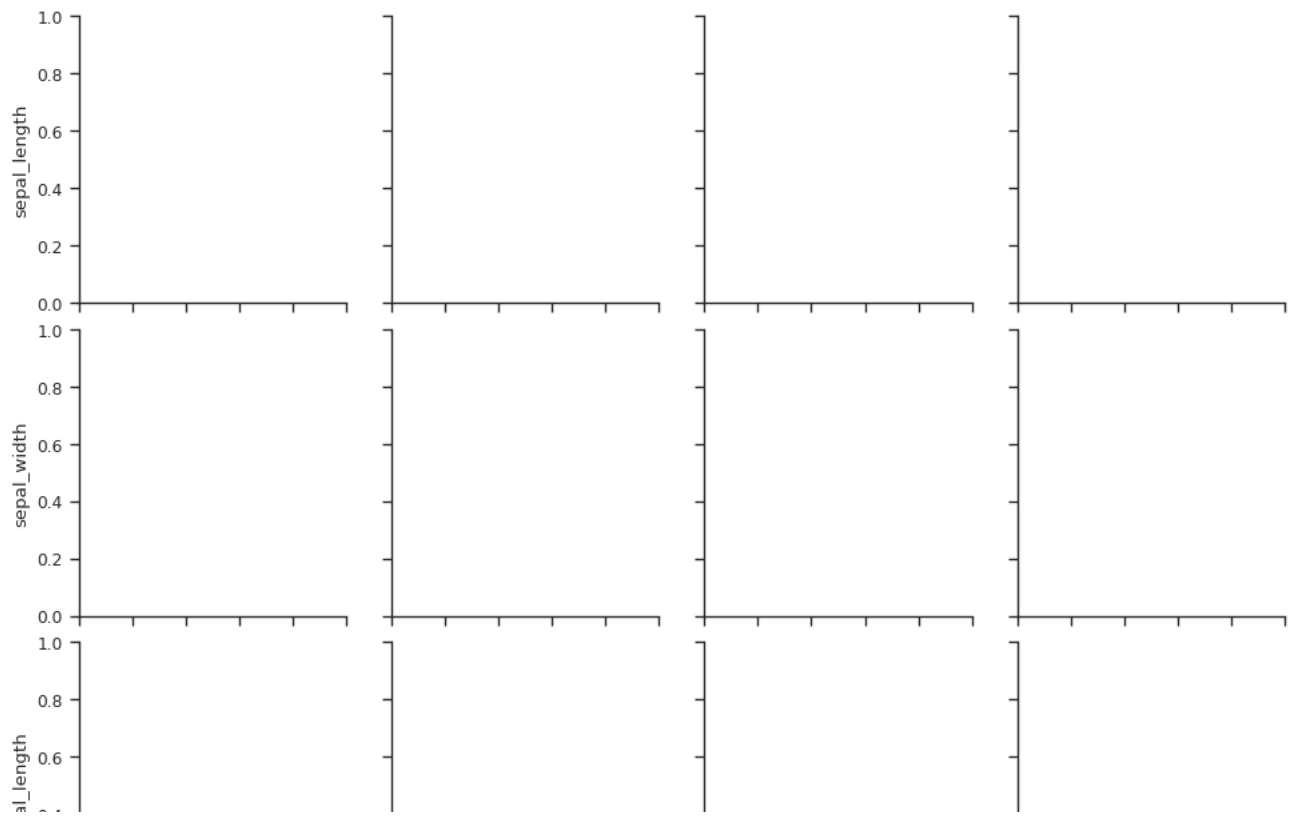


PairGrid

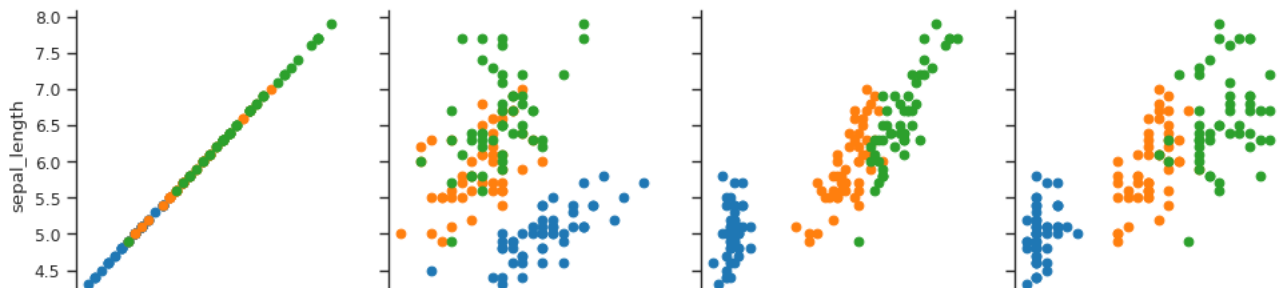
```
iris = sns.load_dataset('iris')
iris.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

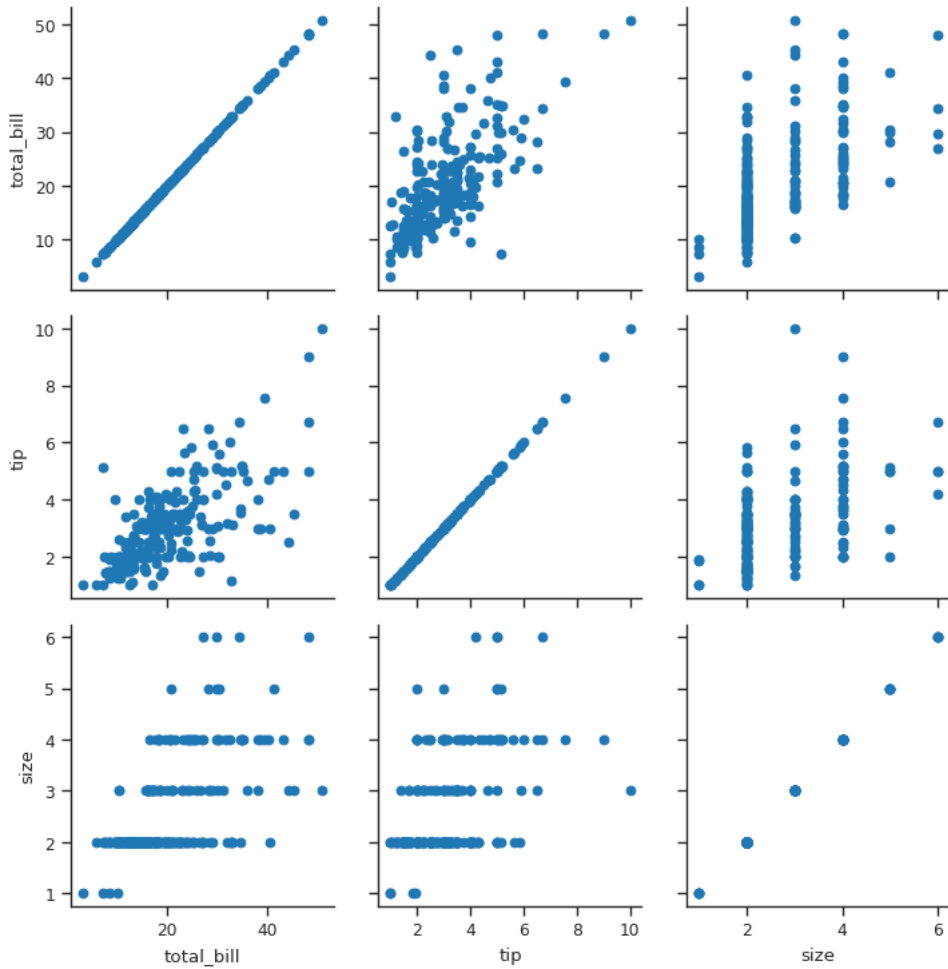
```
iris_grid = sns.PairGrid(iris, hue='species');
```



```
iris_grid = sns.PairGrid(iris, hue='species');  
iris_grid.map(plt.scatter);
```

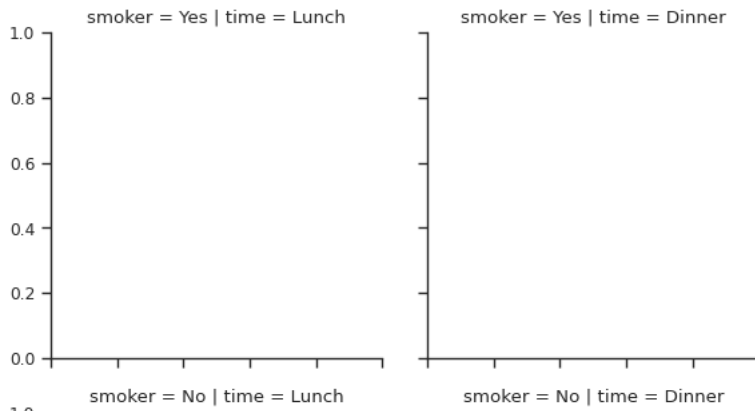


```
tips_grid = sns.PairGrid(tips);
tips_grid.map(plt.scatter);
```

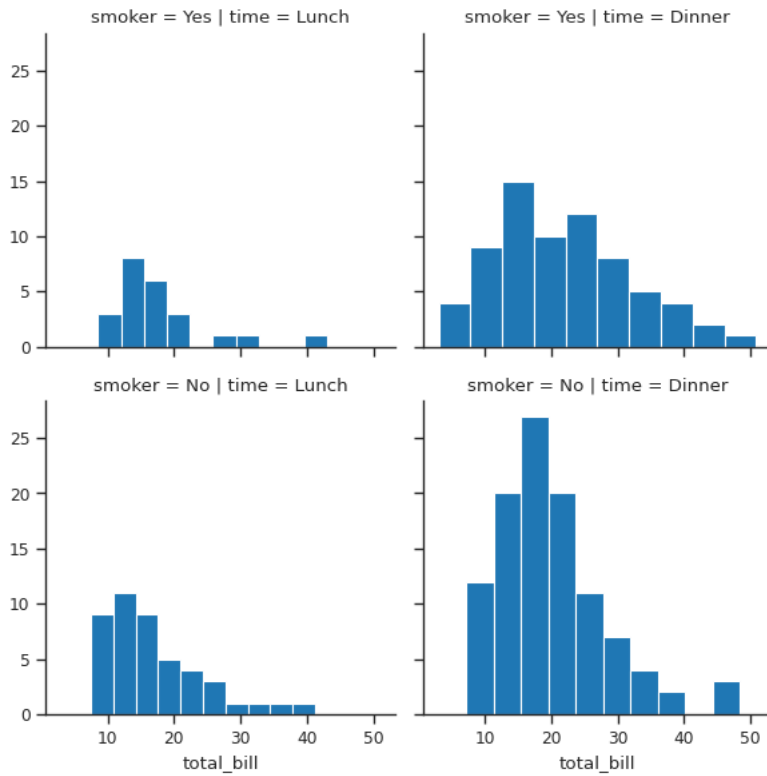


✎ Facet Grid

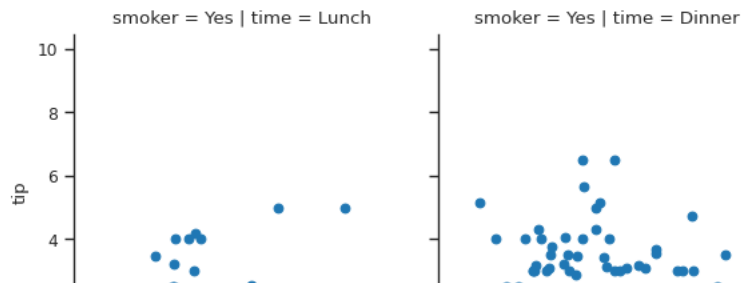
```
tips_fg = sns.FacetGrid(tips, col='time', row='smoker');
```



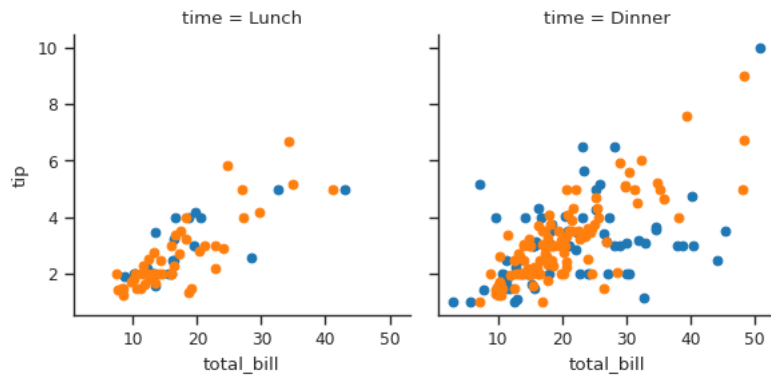
```
tips_fg = sns.FacetGrid(tips, col='time', row='smoker');
tips_fg.map(plt.hist, 'total_bill');
```



```
tips_fg = sns.FacetGrid(tips, col='time', row='smoker');
tips_fg.map(plt.scatter, 'total_bill', 'tip');
```

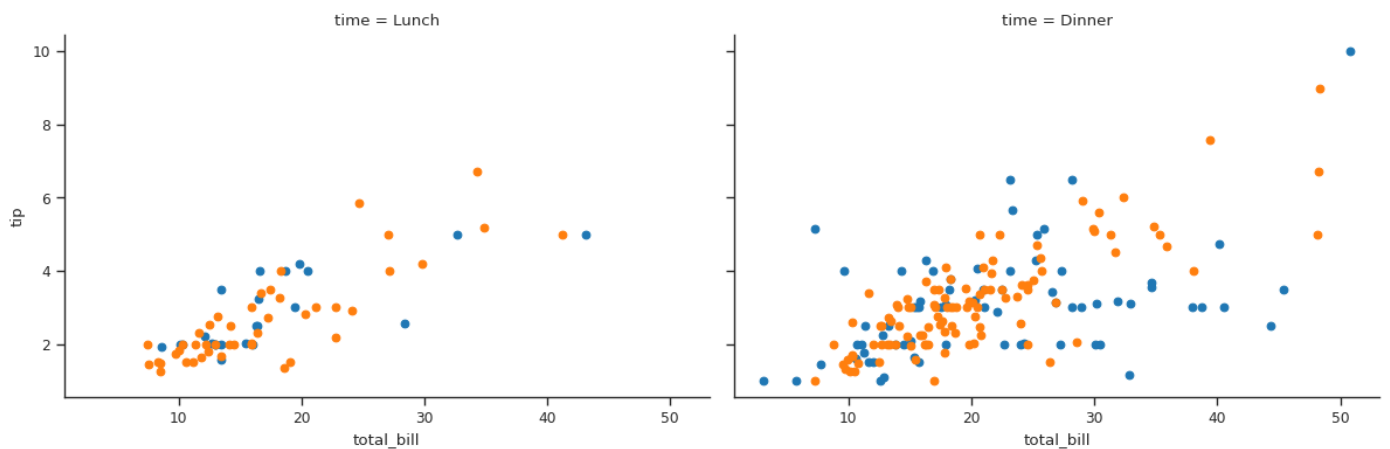



```
tips_fg = sns.FacetGrid(tips, col='time', hue='smoker');
tips_fg.map(plt.scatter, 'total_bill', 'tip');
```



```
tips_fg = sns.FacetGrid(
    tips,
    col='time',
    hue='smoker',
    height=4,
    aspect=1.5
);
```

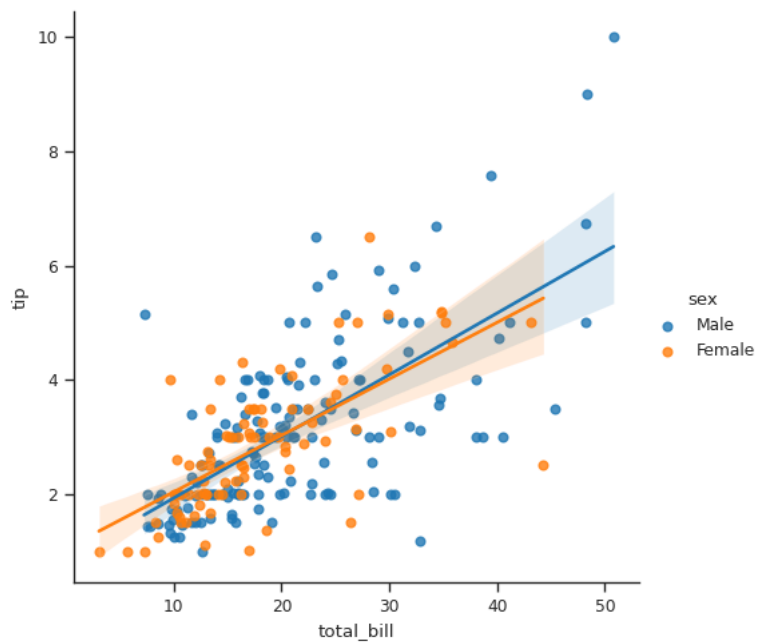
```
tips_fg.map(plt.scatter, 'total_bill', 'tip');
```



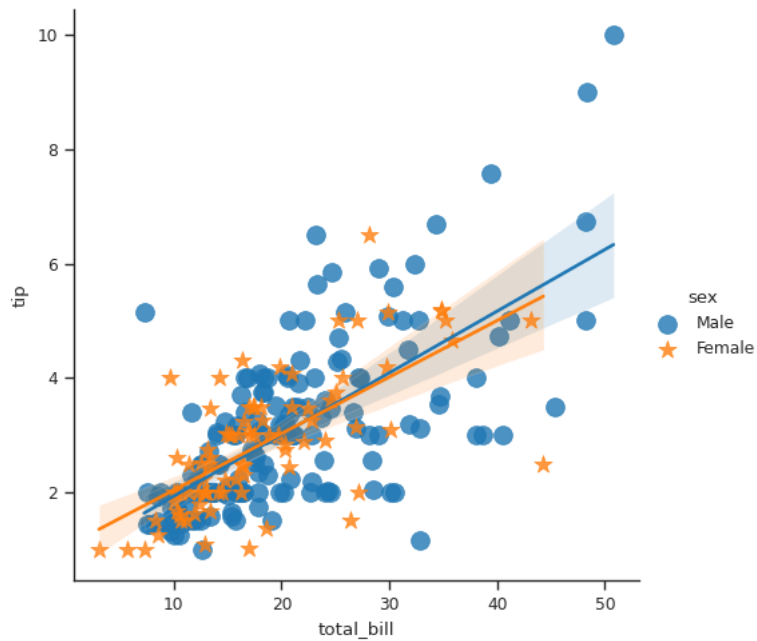
▽ Regression Plots

```
sns.set_context('paper')
```

```
sns.lmplot(data=tips,
           x='total_bill',
           y='tip',
           hue='sex');
```



```
sns.lmplot(data=tips,
           x='total_bill',
           y='tip',
           hue='sex',
           markers=['o', '*'],
           scatter_kws={'s':100, 'linewidth':0.5}
           );
```



```
sns.lmplot(data=tips,
           x='total_bill',
           y='tip',
           col='sex',
           row='time');
```

